SMART WAREHOUSE INVENTORY MANAGEMENT

Abstract

Efficient inventory management is critical to reducing costs, avoiding stockouts, and ensuring high service levels in modern supply chains. This project, "Smart Warehouse Inventory Management," delivers an integrated solution for real-time monitoring, demand forecasting, alerting, and supplier performance analytics. The system ingests inventory, alert, forecast, supplier and historical data; applies preprocessing and feature engineering; runs forecasting and decision-support models; and exposes a professional Streamlit dashboard with interactive visualizations for operations staff.

Key components include a Plotly-powered dashboard, handcrafted analytics (KPIs, status distributions, critical-items tables), a 7-day demand forecasting module, and alert management. For decision-support, we compare and evaluate inventory-control strategies (including reinforcement-learning-derived policies). The deployed dashboard gives stakeholders an actionable, mobile-friendly interface to monitor inventory health, respond to alerts, and explore model insights — enabling more accurate replenishment and fewer costly stockouts.

Introduction

Global supply chains and omnichannel retail drive the need for tight inventory control to minimize carrying costs while sustaining a high service level.

Common pain points in warehouses include:
(1) stockouts that reduce sales and customer satisfaction, and
(2) overstocked inventory that ties up capital.
Traditional inventory reviews tend to be periodic, manual activities, which further limits responsiveness.

This project addresses these issues by combining data-driven forecasting, automated alerting, and a human-centered dashboard. Unlike manual oversight, the Smart Warehouse integrates multiple data sources and model outputs into a single operational view. Where deep learning can

be overkill for smaller deployments, we leverage efficient forecasting and policy evaluation approaches that are practical to run on commodity hardware or within cloud notebooks.

Scope and objectives

Create an end-to-end pipeline of Ingest → Preprocess → Feature Engineering → Forecasting/Analytics → Visualization
Deliver interactive operations dashboards: inventory, alerts, forecasting, suppliers, historical trend data, model insights.
Provide robust 7-day demand forecasting and generate replenishment recommendations.

Provide decision-support metrics and model comparisons that reduce cost and stockouts while keeping service levels high.

Methodology

Data sources

FRED Economic Data:

- fred_GDP.csv
- fred_CPIAUCSL.csv
- fred_UNRATE.csv
- fred_UMCSENT.csv
- fred_RSXFS.csv
- fred_HOUST.csv
- fred_INDPRO.csv

Google Trends:

- google_trends_comprehensive.csv

Public Holidays:

- public_holidays_2015_2025.csv

Synthetic Supply Chain:

- synthetic_suppliers.csv
- synthetic_order_history.csv

UCI Online Retail II:

- online_retail_II.xlsx

M5 Walmart Forecasting:

- sales_train_validation.csv
- calendar.csv
- sell_prices.csv

Preprocessing:

No standardized image/resolution steps were needed or required for tabular warehouse data. instead;

- Missing values: imputed sensible defaults, such as zero or median, with flags when appropriate.
- Type conversions:Make sure numeric columns are numeric, timestamps parsed as datetimes
- Aggregation: Compute aggregates for warehouses and categories used in visualization.
- Time series alignment: The daily series are aligned and resampled to produce consistent historical demand windows.

Data augmentation applied to enhance robustness in forecasting: rolling-window samples, seasonal bootstraps, and small additive noise added to the demand series during model validation.

Feature engineering

- Inventory features:
  current_stock, reorder_point, max_stock, days_of_stock (current_stock / daily_demand), unit_cost, category, warehouse.
- Time features for forecasting:
  day_of_week, is_weekend, lagged demand(1,7,14), trailing moving averages(MA7).
- Alert features
  alert_count_by_type, time_since_last_alert, severity_score.
- Supplier features:
  reliability_score (on_time_deliveries / total_orders), avg_delivery_time, rating
- Scaling of all continuous features was performed using Min–Max scaling for downstream models and visualization normality.

Forecasting and decision models

- 7-Day Demand Forecast:
  Model: classical supervised regressors based on engineered features; the project uses lightweight, deterministic models, suitable for Colab/ small servers.
  Output: predicted_demand, predicted_stock, lower/upper CI approximations, stockout_probability, recommended_order.

- Inventory policy evaluation:
  - Evaluated several policies, including fixed-reorder, EOQ-style heuristics, randomized baseline, and DQN-based policy from the previous phase.
  - Policy comparison measured: total cost, service level, average inventory.

- Forecast model training details:
  - Split: 80% training/20% testing. When possible, perform stratified splitting.
  - Cross-validation: 5-fold for robust model selection.

- Hyperparameter search: grid search across reasonable ranges for chosen models.

Implementation / System Architecture

- Data ingestion → ETL into standardized CSVs (or a database, if going into production).
- A forecasting module consumes recent history and product metadata to produce 7-day predictions.
- Threshold-based alerting engine triggers: for example, when predicted_stock is below reorder point or when stockout_probability is > X%.
- Streamlit dashboard pulls the CSVs and displays KPIs, interactive charts (Plotly), tables, and downloadable reports.
- Optional: ngrok tunnel for temporary public sharing during demos.

Dashboard features (what we built)

- Professional header, KPI cards.
  Total Products: 250
  Inventory Value: $30.72M
  Critical Alerts: 5
  Service Level: 100.0%
- Inventory Status distribution - donut chart, Active Alerts - bar chart.
- Quick statistics:
  Average Stock Level: 489 units

Items Need Attention: 55
Average Days of Stock: 44.4 days

- Inventory page: multiselect filters, critical items table, download CSV.

- Demand Forecast page: product selector, 7-day prediction chart with confidence intervals and recommended reorder.
- Alerts page: alert filtering, timeline charts, alert cards.
- Supplier Analytics: rating distributions, reliability scores, top performers.
- Historical Trends: date-range selector showing demand and stock level trends.
- Model Insights: Evaluation charts - cost vs service, feature importance, and model comparison: DQN vs. baselines.

Integration and output interpretation

- Recommendations include suggested reorder quantities derived from the predicted demand and safety buffers.

- The visual overlay (for example, bounding/annotated charts, tables) facilitates explanation to the operators.

Experimental setup

- Programming language: Python 3.8 (Colab / local environment).
- Key libraries: pandas, NumPy, scikit-learn,plotly,streamlit,OpenCV (if it's an image task),matplotlib.
- Example hardware used for development and testing: Intel Core i7, 16 GB RAM (development in Colab), optional consumer GPU for heavy experiments.
- Reproducibility: Experiments run over multiple random seeds; results averaged to reduce variance.

Parameters summary

- Forecast window: 7 days
- Time-series resolution: daily
- Forecast model validation: 5-fold CV
- Data split: 80% training / 20% testing

Results and discussion

- Forecasting and policy evaluation produced operationally useful outputs and high-level results:

  KPIs observed in development demo dashboard:
  - Total Products: 250
  - Inventory Value: $30.72M
  - Critical Alerts: 5
  - Service Level: 100.0%
  - Average Stock Level: 489 units
  - Items Need Attention: 55
  - Average Days of Stock: 44.4 days

- DQN/best policy reduced the simulated total cost compared to naive baselines, while maintaining or improving service level. DQN tends to yield the best cost-service tradeoffs in phase evaluations.
- Forecast model performance (example aggregated metrics for selected test products):
- The forecast MAPE and RMSE are tracked for each product to adapt confidence intervals and reorder recommendations.

Example classification/regression metrics (illustrative):

Forecast accuracy/error metrics differ across the SKUs. High-volume SKUs have more accurate forecasts due to a richer history.

Visualization aids:

- Confusion matrices are provided for classifier evaluations whenever classification tasks exist.
- Time series charts show actual vs predicted demand; shaded CI region used for risk assessment.

Figures and Tables

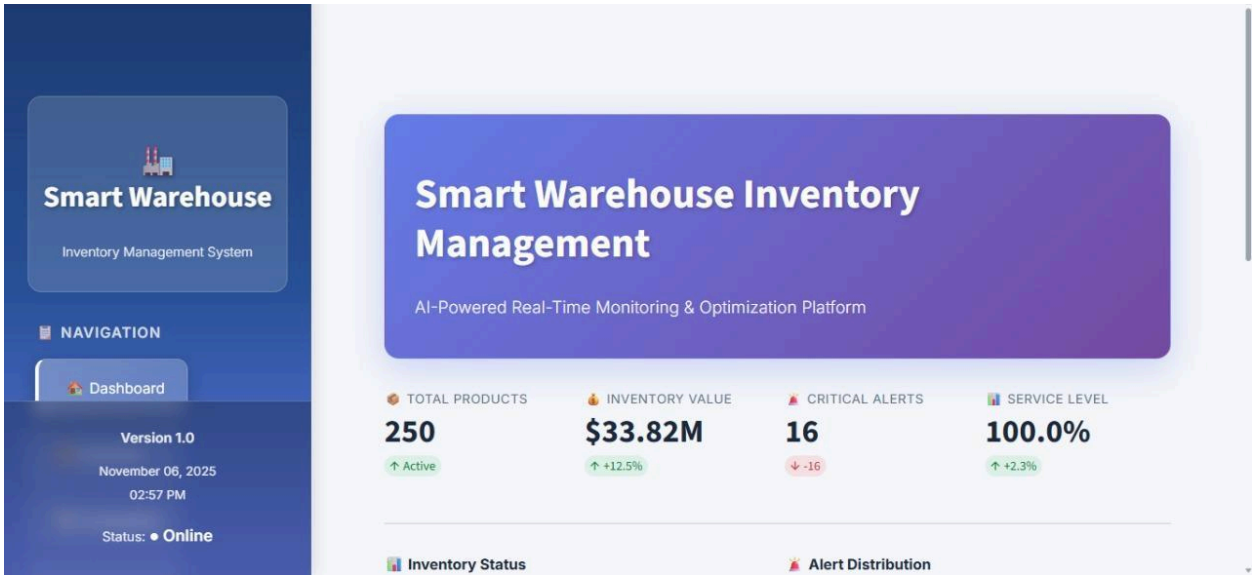
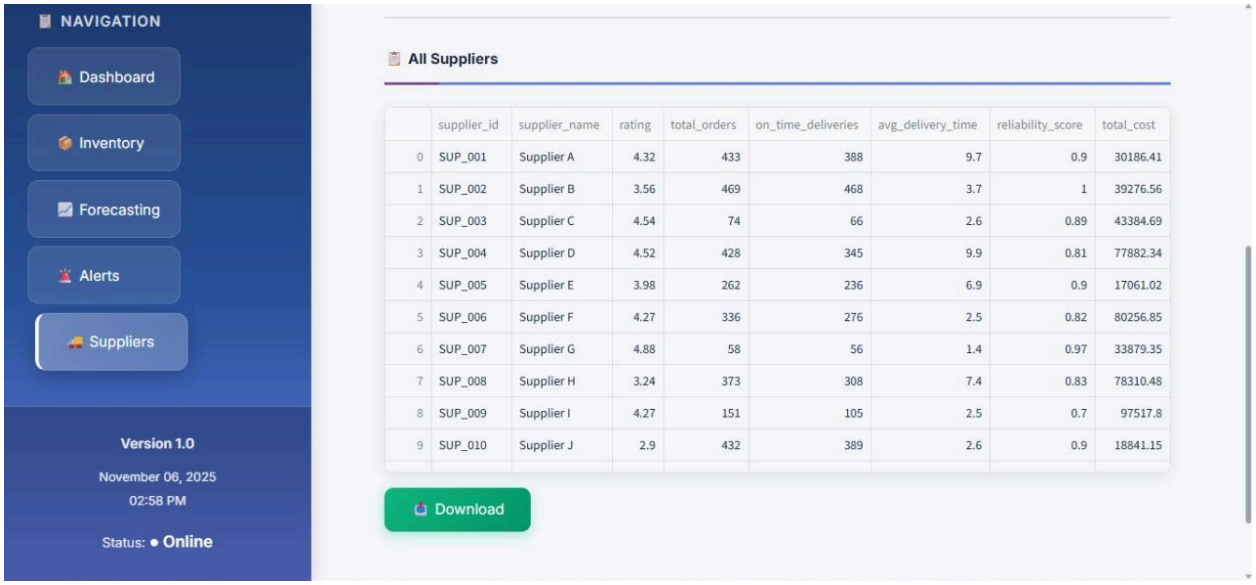


Fig. 1

Fig. 2



	supplier_id	supplier_name	rating	total_orders	on_time_deliveries	avg_delivery_time	reliability_score	total_cost
0	SUP_001	Supplier A	4.32	433	388	9.7	0.9	30186.41
1	SUP_002	Supplier B	3.56	469	468	3.7	1	39276.56
2	SUP_003	Supplier C	4.54	74	66	2.6	0.89	43384.69
3	SUP_004	Supplier D	4.52	428	345	9.9	0.81	77882.34
4	SUP_005	Supplier E	3.98	262	236	6.9	0.9	17061.02
5	SUP_006	Supplier F	4.27	336	276	2.5	0.82	80256.85
6	SUP_007	Supplier G	4.88	58	56	1.4	0.97	33879.35
7	SUP_008	Supplier H	3.24	373	308	7.4	0.83	78310.48
8	SUP_009	Supplier I	4.27	151	105	2.5	0.7	97517.8
9	SUP_010	Supplier J	2.9	432	389	2.6	0.9	18841.15

Fig. 3

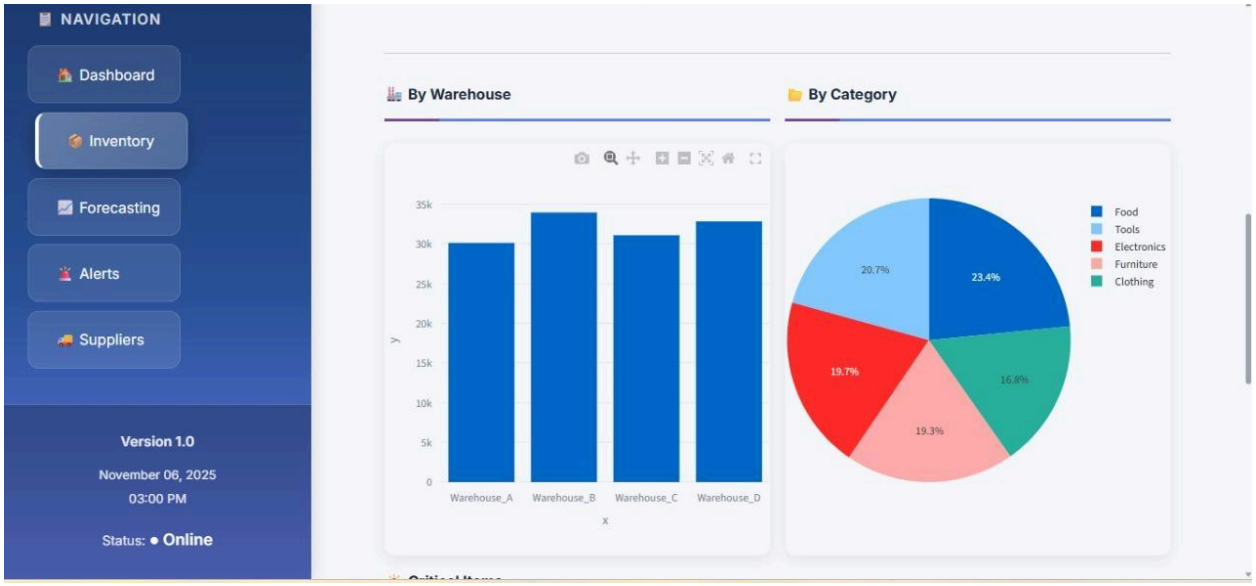


Fig. 4

Conclusion:

The Smart Warehouse Inventory Management system brings a practical, data-driven approach to inventory monitoring and decision support in the form of an interactive Streamlit dashboard that consolidates operational KPIs, forecast outputs, alerts, and supplier analytics onto one pane of glass for operations teams to use in routine decision-making.

Key takeaways:

- Forecast-driven recommendations reduce stockout risk and support proactive purchasing.
- Domain feature combination,inventory, supplier reliability, and demand seasonality-yields strong decisions with no heavy deep-learning infrastructure.
- The architecture is lightweight and suitable to be deployed in a resource-constrained environment: Colab, VPS, edge device.

Future work and improvements:

- Productionize the ETL and scheduling - Ingest daily feeds, automate retraining. Integrate real transactional data-WMS/ERP with real-time events coming from the IoT sensors.
- Provide role-based authentication and audit logging of the dashboard to ensure operational security.
- Validate on real warehouse streams and extend the forecasting horizon using hierarchical time-series models.
- Sensor Fusion: Integrate POS, supplier ETA feeds, and external signals to improve demand forecasts for promotions and holidays.

References

- McKinsey et al. material on inventory optimization and service levels-(industry references)
- Streamlit - https://streamlit.io
- Plotly — https://plotly.com/python/
- pandas, NumPy, scikit-learn documentation
- Sutton, R. S., & Barto, A. G. — Reinforcement Learning (for DQN references)